

ARD-legacy: anchored row diagrams

DEFINITIONS & PROOFS / ENGLISH EDITION

These are the preserved legacy rules; the historical name ARD below means ARD-legacy. For the new default see ARD.

Release definition, 2026-09-13. PDF · NER arc-diagram expander · Python · well-ordering proof · ordinary Lean entry point.

ARD means **Anchored Row Diagrams**. An expression is a finite diagram, not a path of operations. Its row labels are addresses of earlier columns. When a source block is copied, the row anchor moves along with the root, parent and child. Resource guards in the software are not mathematical rules.

1. Finite syntax and root closure

A finite diagram is a string of columns $G = (C_0, \dots, C_{m-1})$. A compressed entry in column j is

$$(k, p, q), \quad 0 \leq k < j, \quad 0 \leq q \leq p < j.$$

The coordinates are the **row anchor**, **parent column**, and **maximum root**, respectively. The child is the column containing the entry. There is no condition $k \leq p$: an anchor may lie after its parent, but always before its child.

The entry abbreviates every atomic relation (k, t, p, j) with $0 \leq t \leq q$. This is downward closure in the root, not downward closure in the anchor. Merge entries with the same (k, p) by keeping only the largest q . Ignore duplicate relations but retain empty columns. The zero diagram has no columns and is written $\emptyset$; $[]$ is one empty column and represents the natural number one. The natural number m consists of m empty columns.

In the lossless list view, each pair of brackets encloses one column; for example

```
[[[(0,0,0)][(1,1,1)]]
```

is the three-column seed A_3 . Python and NER store maximum roots. The Lean finite geometry uses explicit root closure; these are encodings of the same finite relations.

2. Comparison

Within a column, sort the compressed entries in decreasing order of the key (p, k, q) . Compare the resulting lists lexicographically by that key. Compare diagrams from their leftmost column, again lexicographically. A proper prefix is smaller at either level. Thus one only needs the earliest different column and its earliest different entry. No expansion, reachability search, path comparison or global support key is used.

Explicitly listing every root and sorting by (p, k, t) gives the same order: for fixed (p, k) , the largest differing root decides the comparison before any smaller group is reached.

Adjoin an external greatest symbol **Top**, displayed as **Limit** of ARD. It is not a finite column or a row anchor.

3. Fundamental sequences

The index n is a nonnegative integer. Define $0[n] = 0$; this identity is not a strict descent step. For nonempty G , put $x = m - 1$ and let ∂G delete its last column. Set

$$G[0] = \partial G, \quad G[n] = \partial G \quad \text{for all } n \text{ if } C_x = \emptyset.$$

For the remaining case, select the greatest last-column entry by **anchor, root, parent**, the key (k, q, p) , and write it (k, c, r) . This differs from the **parent, anchor, root** comparison key. Put

$$\ell = x - c > 0, \quad \phi_b(i) = \begin{cases} i, & i < c, \\ i + b\ell, & i \geq c. \end{cases}$$

The cut is the control parent c , not $\max(k, c)$. The output has $x + n\ell$ columns. Form the following union of entries, then close downward in the root and normalize.

1. For $b = 0, \dots, n$, copy the old prefix $G \upharpoonright x$. An entry (h, p, q) in column j is sent to $(\phi_b(h), \phi_b(p), \phi_b(q))$ in column $\phi_b(j)$. All four coordinates move, including the anchor.
2. For each $b = 0, \dots, n - 1$, form a seam in column $x + b\ell$. For every old last-column entry (h, p, q) :
 - if $h < k$, insert $(\phi_b(h), \phi_b(p), \phi_b(q))$;
 - if $h = k$, insert $(\phi_b(h), \phi_b(p), \min(\phi_b(q), \phi_b(r) - 1))$, but only if its last coordinate is nonnegative;
 - if $h > k$, insert nothing.
3. In the same seam, insert every entry $(h, \phi_b(c), \phi_b(c))$ with $0 \leq h < \phi_b(k)$.

The unchanged prefix before c is shared by all copies; repeated insertion there does not multiply relations. There are $n + 1$ copies of the source block $[c, x]$, with n seams. The first column of a new copy and the preceding seam occupy the same column, so their entries are merged.

Moving a maximum root across a gap and then taking root closure also fills roots in that gap. One must not replace this rule by transporting only the old individual roots and omitting the newly intervening ones.

Every call consists of finite loops and finite coordinate operations. It does not recursively expand a newly produced graph. Finite definition does not imply a small output or a practical running time.

4. Seeds, standard domain and structural properties

Define

$$A_0 = 0, \quad A_1 = [\emptyset], \quad A_{n+1} = A_n \curvearrowright [(n-1, n-1, n-1)] \quad (n \geq 1), \quad \mathbf{Top}[n] = A_n.$$

Let $D(H)$ contain H and all graphs obtainable from it by a finite expansion path, ignoring the $0 \rightarrow 0$ self-loop. The finite standard domain is

$$U = \bigcup_{n < \omega} D(A_n).$$

The complete notation is $U \cup \{\mathbf{Top}\}$. A constructor or parser only checks structural legality; it does not certify membership in U .

Expansion preserves structural legality. For every finite G , $G[n]$ is a complete-column prefix of $G[n+1]$; it is proper if the old last column is nonempty. All columns before the old last column are unchanged. Every nonzero expansion strictly decreases the specified column order. Repeated index-zero steps obtain every finite prefix.

The linked paper proves, in $S = KP_\omega +$ “there exists an uncountable ordinal”, that all structurally legal finite graphs admit an ordinal-valued rank decreasing at every nonzero expansion, and that column order is a well-order on U , with or without the external greatest element. Here KP includes **full Set Induction**, but not Power Set or Choice. The ordinary Lean entry point is separate from an encoding of a derivation in S ; see the paper’s verification discussion.

The standard-domain restriction on the column-order theorem is essential. The full legal graph space contains the descending chain

$$[\emptyset]^{m+1}[(0,0,0)] > [\emptyset]^{m+2}[(0,0,0)] > \dots$$

This is not an expansion chain. No comparison of ARD's order type with Y, RPD, LRD or Ω -LRD3, and no optimal axiomatic strength, is asserted here.

5. Counts and displays

The NER file provides a lossless list view, a count-sequence view and a full arc-diagram view. The count sequence is only a display: it is not an injective encoding, an inverse input syntax or the comparison key.

For one column, keep its left prefix fixed, treat it as last, perform a positive-index expansion and discard the newly appended columns. Repeat this local operation until the retained column has no entries, then delete it. Its count is the number of local operations, including that final deletion. An empty column has count one. At the retained position only the zeroth seam and the source cut column matter, so the local operation is independent of which positive index was used. Whenever a positive-index expansion rewrites a nonempty last column, the count at that position decreases by exactly one.

The local finite count computation terminates; it is not a simulation of the entire repeated [1] trajectory. The count sequences of $A_1, \dots, A_6$ are the successive prefixes of 1, 2, 6, 23, 104, 537. Counts do not establish global well-ordering on their own.

The arc view shows the complete graph rather than a cropped piece. Exact counts are calculated with BigInt. If count or drawing limits are reached, the interface reports the limit and points back to the list; it does not silently substitute an approximate count or a partial graph.

6. Software use

Import ARD-arcs.ne-rewritten.js through NER's custom-notation control and select ARD-legacy. Its current registration ID is ard-adjacency-v01; the ordinary list is the default, with 计数序列, 弧线图, 邻接表 (文字) and 邻接表 (图) in the equivalent-display menu. The script's resource safeguards do not change the finite mathematical rule.

The independent Python definition requires Python 3.10+ and only the standard library:

```
from ard import AnchoredRows, TOP

a = AnchoredRows.seed(3)
assert str(a) == "[[(0,0,0)][(1,1,1)]"
assert TOP[3] == a
assert a[0] == AnchoredRows(a.columns[:-1])
assert a[1] < a
assert a[1].columns == a[2].columns[:len(a[1].columns)]
```

AnchoredRows() is zero, AnchoredRows.finite(n) has n empty columns, and TOP is AnchoredRows(None). The constructor normalizes the entries and validates their coordinates. kind is zero, successor or limit; a finite nonzero term is a successor exactly when its last column is empty. $fs(n)$ and $[n]$ are identical. Natural-number inputs reject negative integers and booleans.

The Python definition omits parsing, counts, drawing, caches and resource guards. Its integers have arbitrary precision, but a large expansion can still exhaust time or memory. It should not be used for unbounded brute-force descent searches.